

Chapter 4. Edge Detection

Contents

4.1 Introduction

4.2 Edge detection

4.3 Canny detector

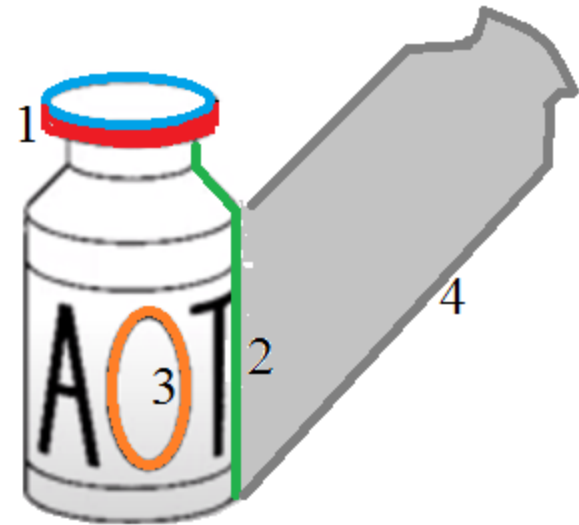
4.4 Corner detection

Chapter 4. Edge Detection

4.1 Introduction

The rapid changes in image intensity are caused by various physical phenomena:

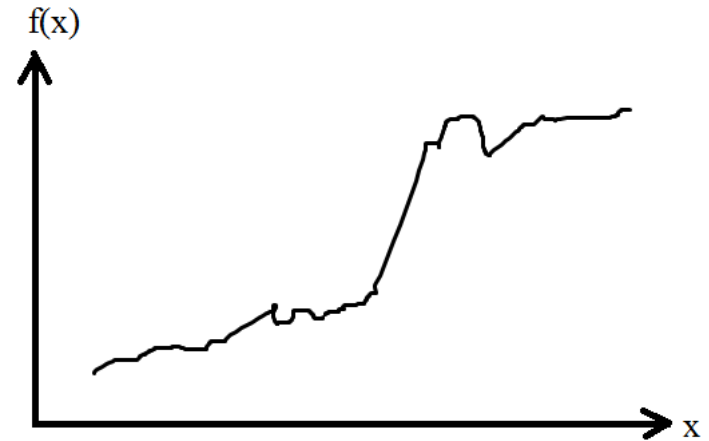
- Surface normal discontinuity
- Depth discontinuity
- Surface reflectance discontinuity
- Illumination discontinuity



Chapter 4. Edge Detection

4.1 Introduction

In real images, the profile in image appears as indicated by the following image:



The main objective is to find:

- The edge position (sub pixel)
- The edge magnitude
- The edge orientation

The required performances:

- High detection rate (find an edge if it exist and not find an edge when it does not)
- Good localisation
- Low noise sensitivity (resilience to noise)

Chapter 4. Edge Detection

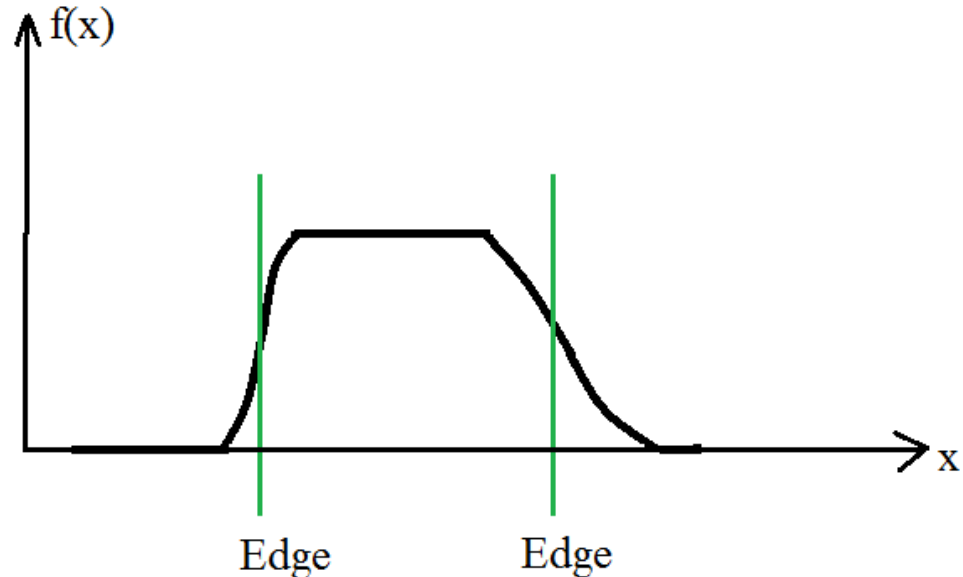
4.2 Edge detection

1D Edge detection using gradient

Edge is a rapid change in image intensity in a small region.

To locate the edge position, we perform of the derivative of the continuous function:

Amount of change in the function.



Chapter 4. Edge Detection

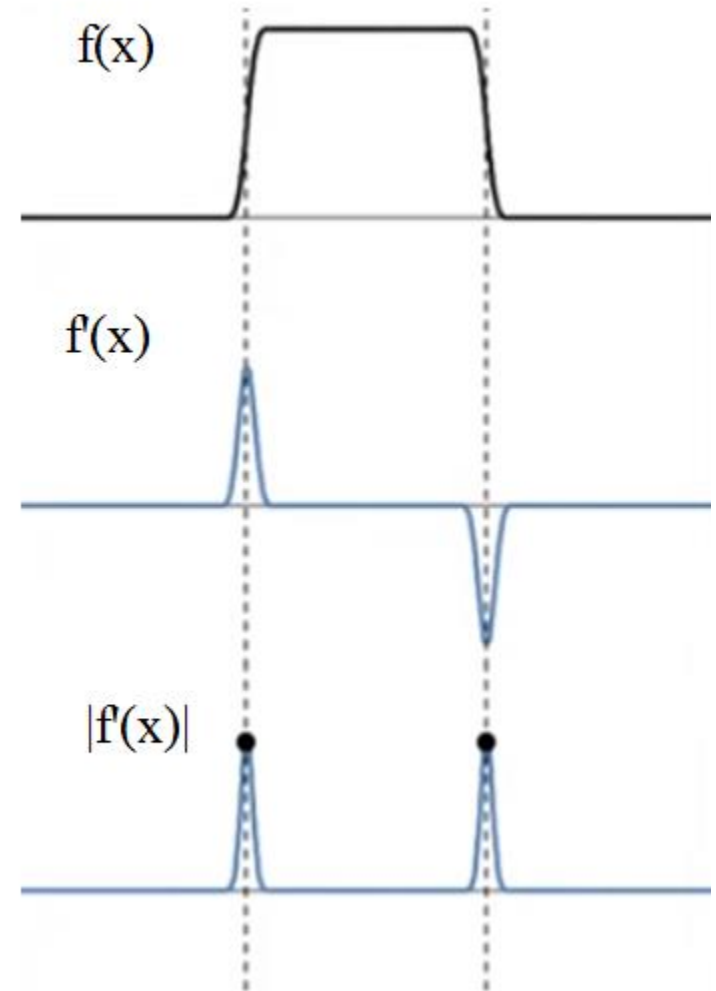
4.2 Edge detection

1D Edge detection using gradient

Using the first derivative provides both location and strength of an edge

Local extrema indicate edges

Local Maxima indicate edges

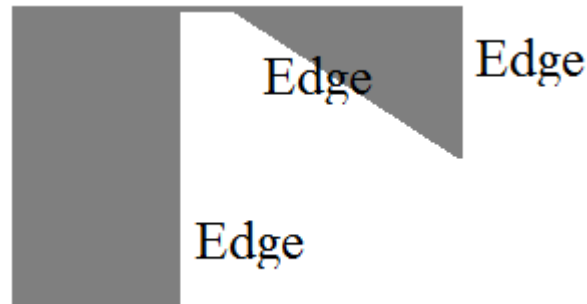


Chapter 4. Edge Detection

4.2 Edge detection

2D Edge detection using gradient

For 2D, partial derivative of 2D continuous function represents the amount of change in each direction.

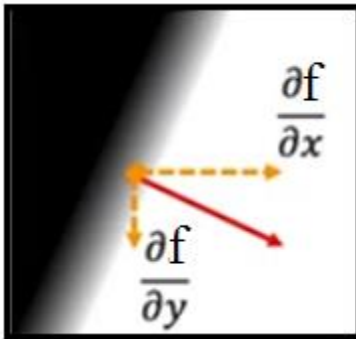


Chapter 4. Edge Detection

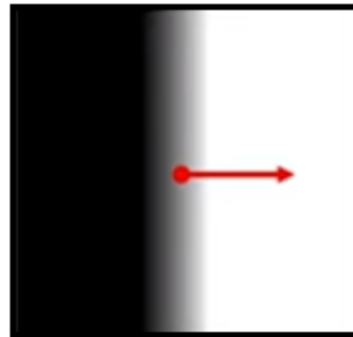
4.2 Edge detection

2D Edge detection using gradient

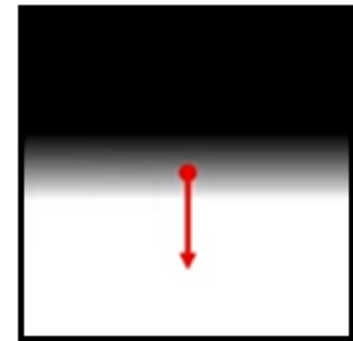
$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$



$$\nabla f = \left[\frac{\partial f}{\partial x}, 0 \right]$$



$$\nabla f = \left[0, \frac{\partial f}{\partial y} \right]$$

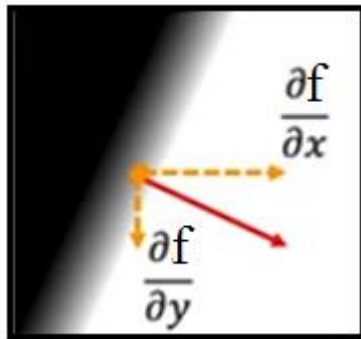


Chapter 4. Edge Detection

4.2 Edge detection

2D Edge detection using gradient Gradient magnitude

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right] \quad \text{Gradient magnitude } S = \|\nabla f\| = \sqrt{\frac{\partial f^2}{\partial x} + \frac{\partial f^2}{\partial y}}$$



Gradient orientation $\theta = \tan^{-1}\left(\frac{\frac{\partial f}{\partial y}}{\frac{\partial f}{\partial x}}\right)$



Orientation make features robust to illumination and rotation variations, enabling better object recognition and tracking.

Chapter 4. Edge Detection

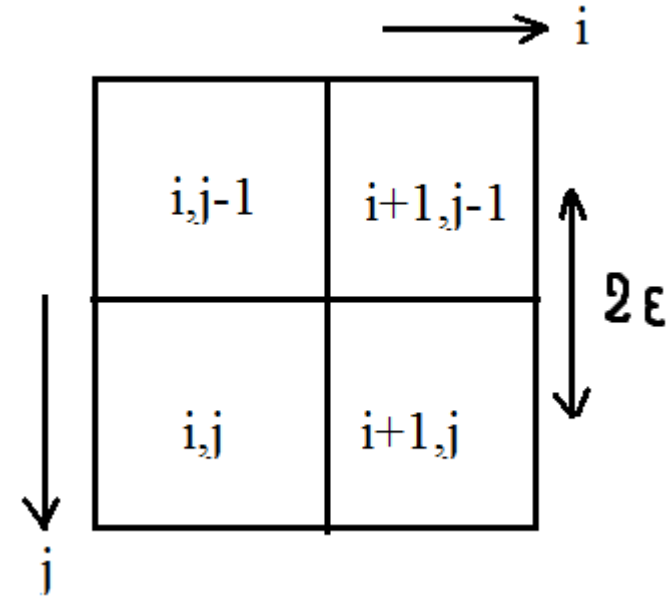
4.2 Edge detection

2D Edge detection using gradient

Discrete Gradient

$$\frac{\partial f}{\partial x} \approx \frac{1}{2\varepsilon} ((f_{i+1,j-1} - f_{i,j-1}) + (f_{i+1,j} - f_{i,j}))$$

$$\frac{\partial f}{\partial y} \approx \frac{1}{2\varepsilon} ((f_{i+1,j-1} - f_{i+1,j}) + (f_{i,j-1} - f_{i,j}))$$



$$\frac{\partial f}{\partial x} \approx \frac{1}{2\varepsilon} \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}$$

$$\frac{\partial f}{\partial y} \approx \frac{1}{2\varepsilon} \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}$$

$\frac{1}{2\varepsilon}$ Correspond à la taille du pixel et sera considérée égale à une unite.

Chapter 4. Edge Detection

4.2 Edge detection

2D Edge detection using gradient

Quelques filtres très utilisés

Gradient	Roberts	Prewitt	Sobel (3x3)	Sobel (5x5)																																															
$\frac{\partial I}{\partial x}$	<table><tr><td>0</td><td>1</td></tr><tr><td>-1</td><td>0</td></tr></table>	0	1	-1	0	<table><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr></table>	-1	0	1	-1	0	1	-1	0	1	<table><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-2</td><td>0</td><td>2</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr></table>	-1	0	1	-2	0	2	-1	0	1	<table><tr><td>-1</td><td>-2</td><td>0</td><td>2</td><td>1</td></tr><tr><td>-2</td><td>-3</td><td>0</td><td>3</td><td>2</td></tr><tr><td>-3</td><td>-5</td><td>0</td><td>5</td><td>3</td></tr><tr><td>-2</td><td>-3</td><td>0</td><td>3</td><td>2</td></tr><tr><td>-1</td><td>-2</td><td>0</td><td>2</td><td>1</td></tr></table>	-1	-2	0	2	1	-2	-3	0	3	2	-3	-5	0	5	3	-2	-3	0	3	2	-1	-2	0	2	1
0	1																																																		
-1	0																																																		
-1	0	1																																																	
-1	0	1																																																	
-1	0	1																																																	
-1	0	1																																																	
-2	0	2																																																	
-1	0	1																																																	
-1	-2	0	2	1																																															
-2	-3	0	3	2																																															
-3	-5	0	5	3																																															
-2	-3	0	3	2																																															
-1	-2	0	2	1																																															
$\frac{\partial I}{\partial y}$	<table><tr><td>1</td><td>0</td></tr><tr><td>0</td><td>-1</td></tr></table>	1	0	0	-1	<table><tr><td>1</td><td>1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>-1</td><td>-1</td><td>-1</td></tr></table>	1	1	1	0	0	0	-1	-1	-1	<table><tr><td>1</td><td>2</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>-1</td><td>-2</td><td>-1</td></tr></table>	1	2	1	0	0	0	-1	-2	-1	<table><tr><td>1</td><td>2</td><td>3</td><td>2</td><td>1</td></tr><tr><td>2</td><td>3</td><td>5</td><td>3</td><td>2</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>-2</td><td>-3</td><td>-5</td><td>-3</td><td>-2</td></tr><tr><td>-1</td><td>-2</td><td>-3</td><td>-2</td><td>-1</td></tr></table>	1	2	3	2	1	2	3	5	3	2	0	0	0	0	0	-2	-3	-5	-3	-2	-1	-2	-3	-2	-1
1	0																																																		
0	-1																																																		
1	1	1																																																	
0	0	0																																																	
-1	-1	-1																																																	
1	2	1																																																	
0	0	0																																																	
-1	-2	-1																																																	
1	2	3	2	1																																															
2	3	5	3	2																																															
0	0	0	0	0																																															
-2	-3	-5	-3	-2																																															
-1	-2	-3	-2	-1																																															

Good → Poor localization

Sensitive → Less sensitive to noise

Poor → Good detection

Chapter 4. Edge Detection

4.2 Edge detection

2D Edge detection using gradient

Edge thresholding

Standard: (Single Threshold T)

$$||\nabla f(x, y)|| < T \text{ *Not an Edge*}$$

$$||\nabla f(x, y)|| < T \text{ *An Edge*}$$

L'hystérésis est un terme utilisé pour décrire un phénomène dans lequel la réponse ou l'état d'un système dépend non seulement des conditions actuelles, mais aussi de son historique.

Hysteresis Based: (Two Thresholds $T_0 < T_1$)

$$||\nabla f(x, y)|| < T_0 \text{ *Not an Edge*}$$

$$||\nabla f(x, y)|| \geq T_1 \text{ *An Edge*}$$

$T_1 > ||\nabla f(x, y)|| \geq T_0$ Is an edge if a neighboring pixel is an edge

Chapter 4. Edge Detection

4.2 Edge detection

2D Edge detection using Laplacian

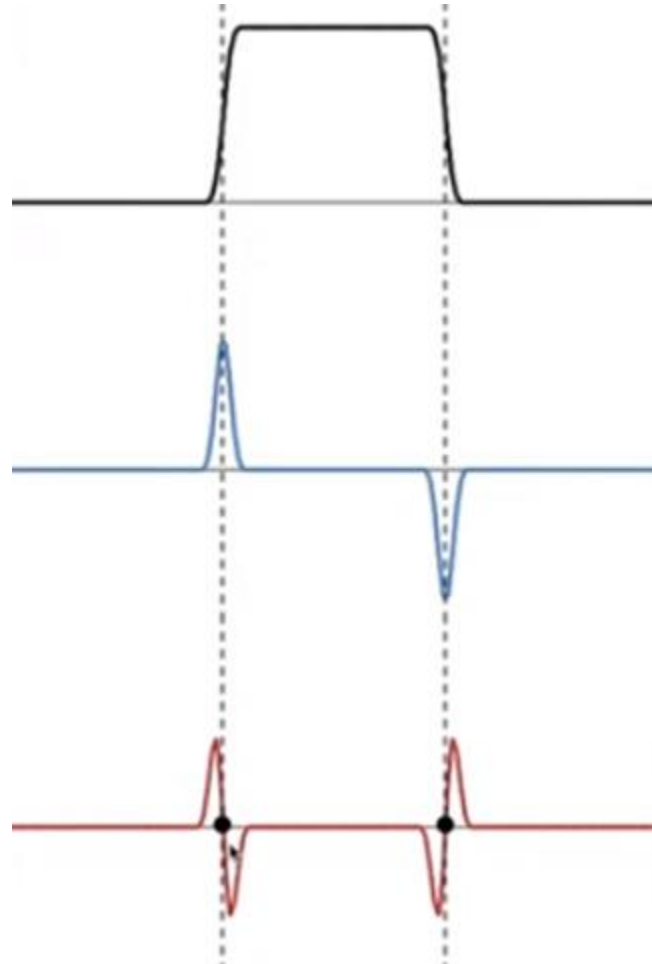
Local extrema

$$f(x)$$

$$\frac{\partial f}{\partial x}$$

Zero Crossing

$$\frac{\partial^2 f}{\partial x^2}$$



Chapter 4. Edge Detection

4.2 Edge detection

2D Edge detection using Laplacian

The Laplacian is the sum of second derivatives.

Edges are “zero crossing” in Laplacian of image.

The Laplacian does not provide directions of edges.

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

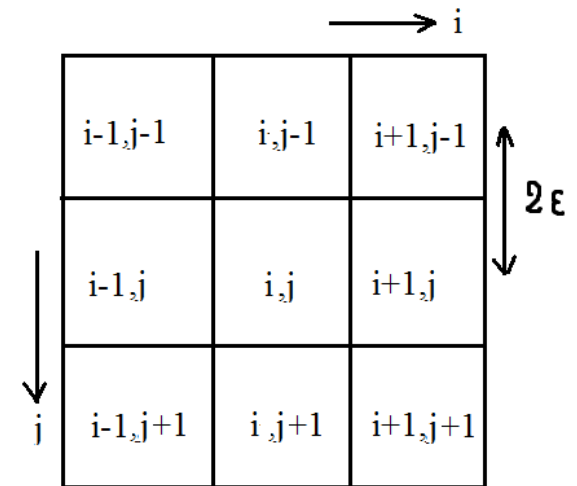
Chapter 4. Edge Detection

4.2 Edge detection

2D Edge detection using Laplacian

$$\frac{\partial^2 f}{\partial x^2} \approx \frac{1}{\epsilon^2} (f(i-1, j) - 2f(i, j) + f(i+1, j))$$

$$\frac{\partial^2 f}{\partial y^2} \approx \frac{1}{\epsilon^2} (f(i, j) - 2f(i, j+1) + f(i, j+2))$$



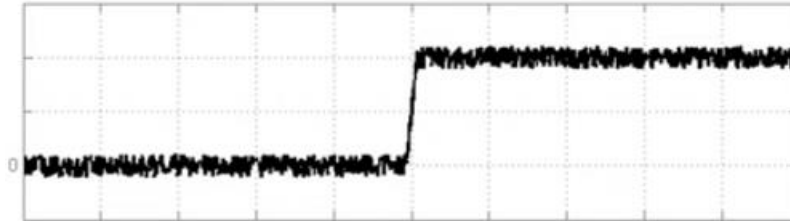
	1	
1	-4	1
	1	

Chapter 4. Edge Detection

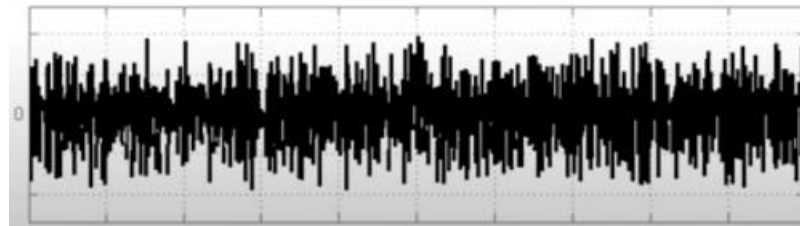
4.2 Edge detection

Case of noisy data

$f(x)$



$\nabla f(x)$

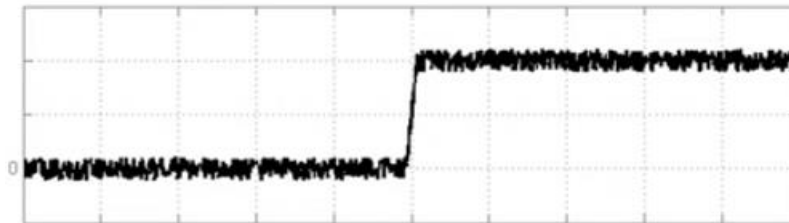


Chapter 4. Edge Detection

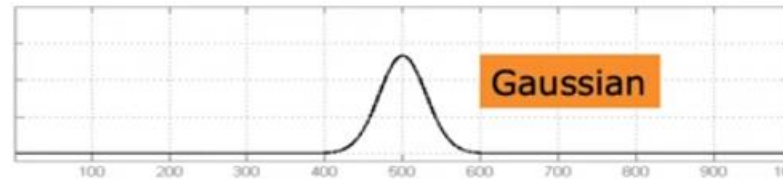
4.2 Edge detection

Case of noisy data

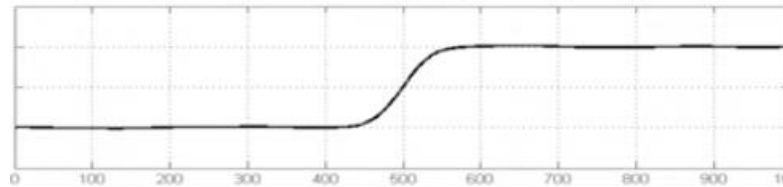
$f(x)$



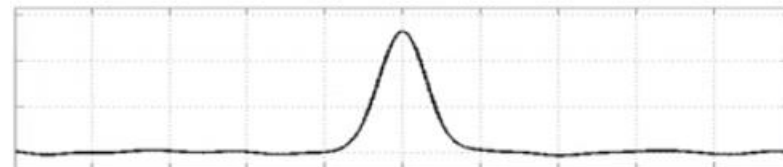
n_σ



$n_\sigma * f(x)$



$\nabla(n_\sigma * f(x))$

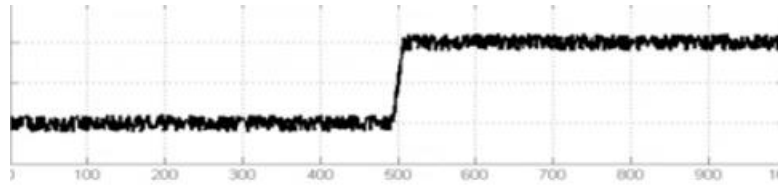


Chapter 4. Edge Detection

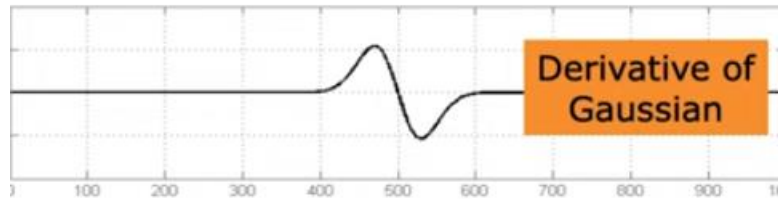
4.2 Edge detection

Case of noisy data

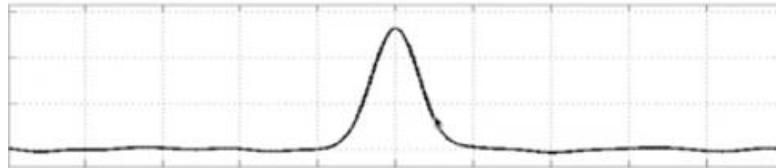
$f(x)$



$\nabla(n_\sigma)$



$\nabla(n_\sigma) * f(x)$



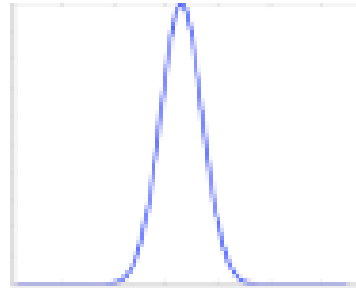
$$\nabla(n_\sigma * f(x)) = \nabla(n_\sigma) * f(x)$$

Chapter 4. Edge Detection

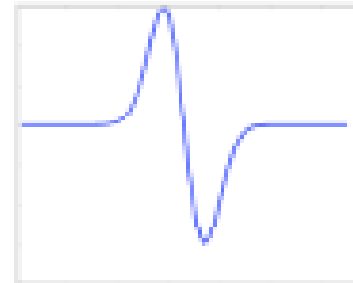
4.2 Edge detection

Case of noisy data

$$g(x) = e^{-\frac{x^2}{2\sigma^2}}$$



$$g'(x) = -\frac{1}{2\sigma^2} 2xe^{-\frac{x^2}{2\sigma^2}} = -\frac{x}{\sigma^2} e^{-\frac{x^2}{2\sigma^2}}$$



$$g''(x) = \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2}\right) e^{-\frac{x^2}{2\sigma^2}}$$

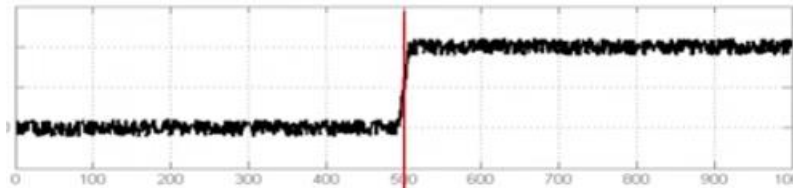


Chapter 4. Edge Detection

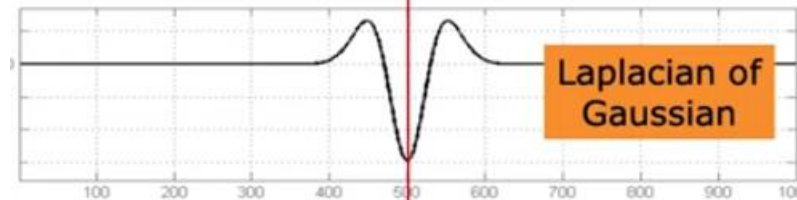
4.2 Edge detection

Case of noisy data

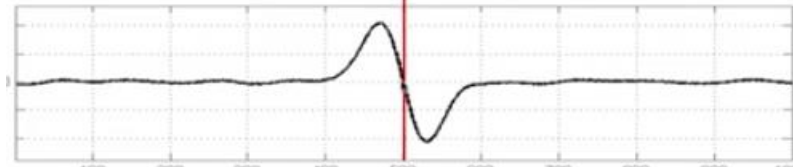
$f(x)$



$\nabla^2(n_\sigma)$



$\nabla^2(n_\sigma) * f(x)$



$$\nabla^2(n_\sigma * f(x)) = \nabla^2(n_\sigma) * f(x)$$

Chapter 4. Edge Detection

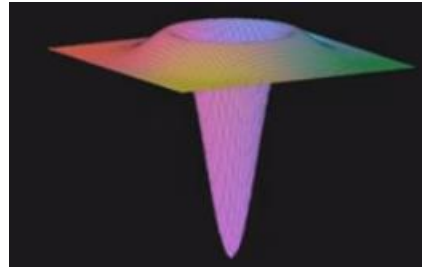
4.2 Edge detection

Case of noisy data

$$\nabla^2(n_\sigma * f(x)) = \nabla^2(n_\sigma) * f(x)$$

$$\nabla^2(n_\sigma) = \frac{\partial^2 n_\sigma}{\partial x^2} + \frac{\partial^2 n_\sigma}{\partial y^2}$$

$$\nabla^2(n_\sigma) * f(x)$$



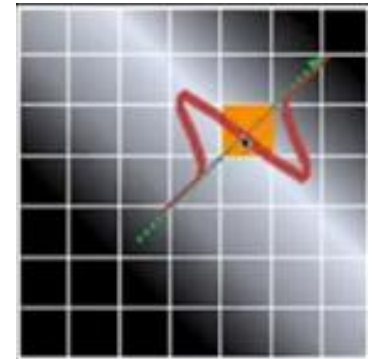
Chapter 4. Edge Detection

4.3 Canny detector

J. Canny, "A Computational Approach to Edge Detection," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. PAMI-8, no. 6, pp. 679-698, Nov. 1986, doi: 10.1109/TPAMI.1986.4767851.

- 1- Smooth the image with 2D Guassian $n_\sigma * f$
- 2- Compute image gradient using Sobel Operator $\nabla n_\sigma * f$
- 3- Find the gradient magnitude at each pixel $||\nabla n_\sigma * f||$
- 4- Find the gradient orientation at each pixel:

$$\hat{n} = \frac{\nabla n_\sigma * f}{||\nabla n_\sigma * f||}$$



$$||\nabla n_\sigma * f||$$

- 5- Compute the Laplacian along the Gradient direction $\hat{n}$ at each pixel

$$\frac{\sigma^2(n_\sigma * f)}{\partial \hat{n}^2}$$

- 6- Find zero crossing in Laplacian to find the edge location

Chapter 4. Edge Detection

4.3 Canny detector

Non-maximum suppression

It works by examining each pixel's gradient magnitude and orientation and comparing it with the neighboring pixels along the gradient direction.

If the central pixel's gradient magnitude is the largest among its neighbors, it means that this pixel is likely part of an edge, and we keep it.

If not, we suppress it by setting its intensity to zero and removing it from consideration as an edge pixel.

Chapter 4. Edge Detection

4.3 Canny detector



Image



$\sigma = 1$



$\sigma = 2$

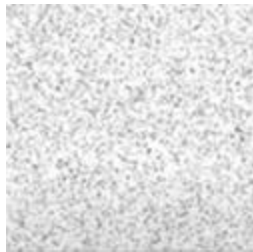


$\sigma = 4$

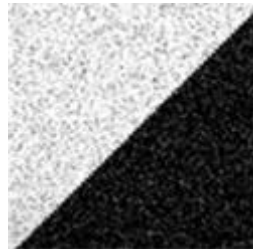
Chapter 4. Edge Detection

4.4 Corner detection

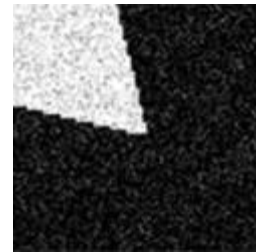
Corner: Point where two edges meet: rapid changes of image intensity in two directions within a small region.



Flat region



Edge region

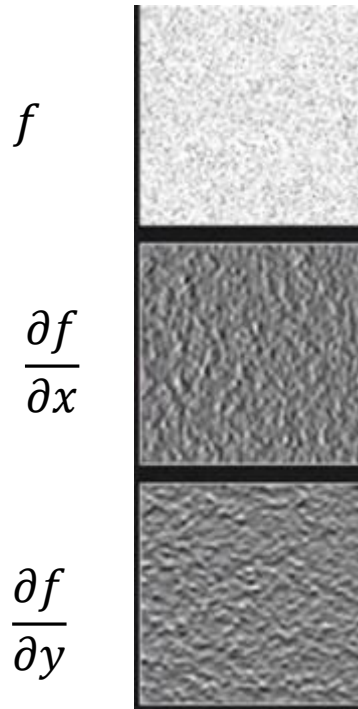


Corner region

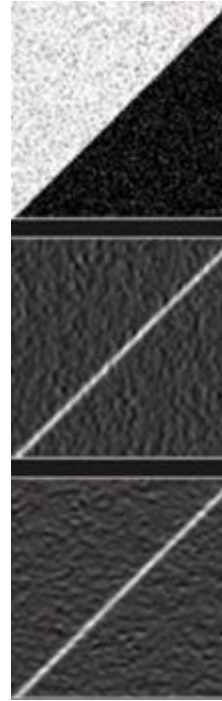
Chapter 4. Edge Detection

4.4 Corner detection

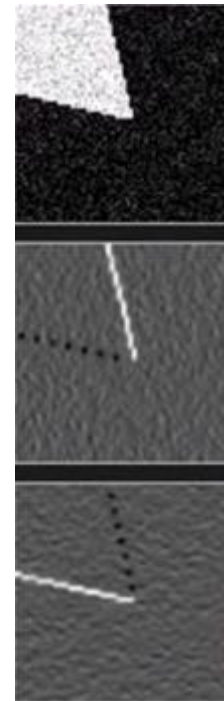
Image gradient



Flat region



Edge region



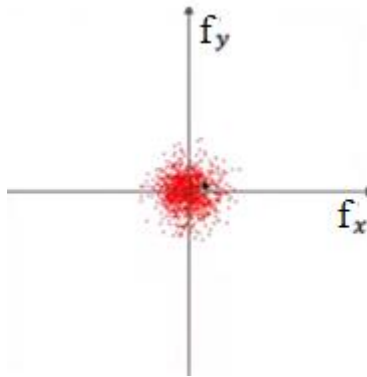
Corner region

Chapter 4. Edge Detection

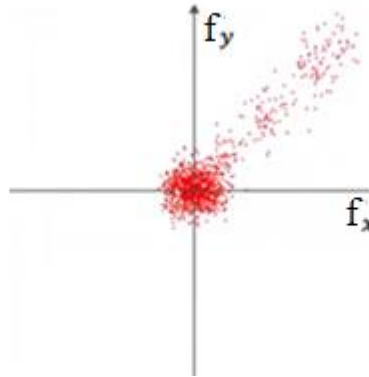
4.4 Corner detection

Distribution of image gradient

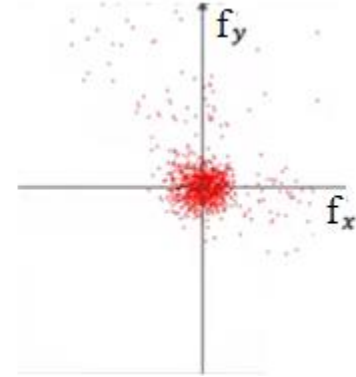
$$f_x = \frac{\partial f}{\partial x} \text{ and } f_y = \frac{\partial f}{\partial y}$$



Flat region



Edge region



Corner region

Chapter 4. Edge Detection

4.4 Corner detection

Harris Detector: Mathematics

A corner may be recognized in a window W as follow:

The considered pixel is the centre of a window W .

The expression of the intensity change when we shift the widow W by (u,v) is given by the following equation.

$$E(u, v) = \sum_{x,y} w(x, y) [f(x + u, y + v) - f(x, y)]^2$$

$w(x,y)=1$ inside the window and 0 outside of the window

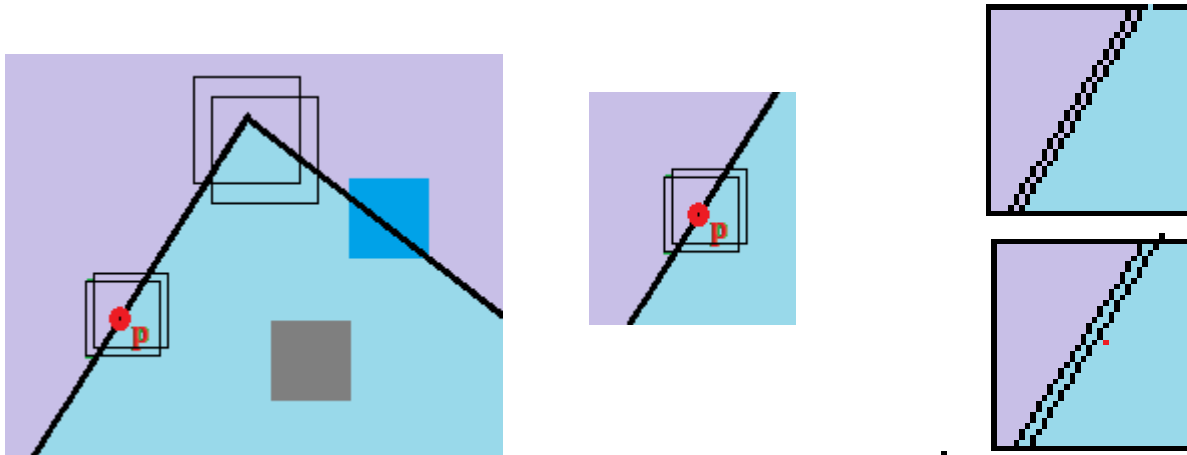
Or be considered as the value of Gaussian inside of the window.

Chapter 4. Edge Detection

4.4 Corner detection

Harris Detector: Mathematics

For example, after the shift of the window (see figure), all pixels values are identical, except those between the two oblique lines.

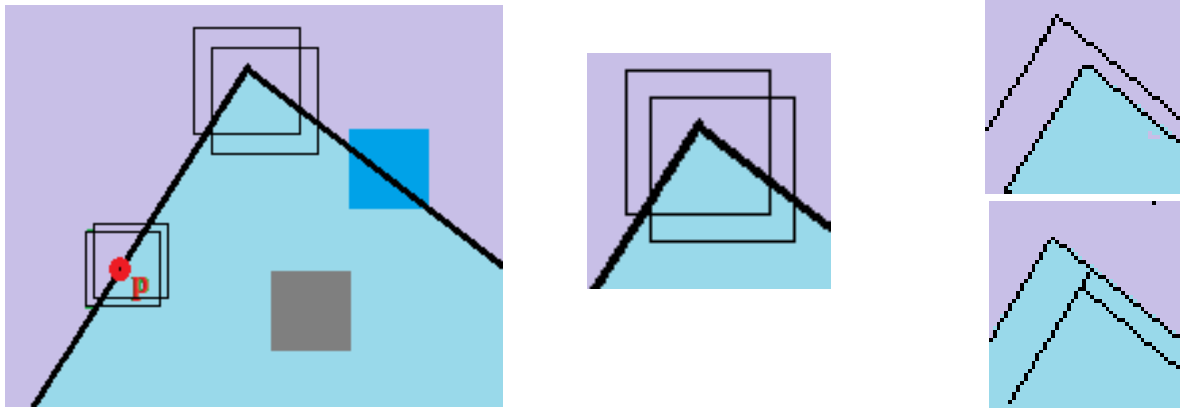


Chapter 4. Edge Detection

4.4 Corner detection

Harris Detector: Mathematics

For example, after the shift of the window towards the corner (see figure), all pixels values are identical, except those between the two pairs of oblique lines. In this case the value of $E(u,v)$ is greater than the previous case. Then, we need to find (u,v) such that $E(u,v)$ is large.



Chapter 4. Edge Detection

4.4 Corner detection

Taylor series for 2D functions

$$f(x+u, y+v) = f(x, y) + uf_x(x, y) + vf_y(x, y) +$$

First partial derivatives

$$\frac{1}{2!} [u^2 f_{xx}(x, y) + uv f_{xy}(x, y) + v^2 f_{yy}(x, y)] +$$

Second partial derivatives

$$\frac{1}{3!} [u^3 f_{xxx}(x, y) + u^2 v f_{xxy}(x, y) + uv^2 f_{xyy}(x, y) + v^3 f_{yyy}(x, y)]$$

Third partial derivatives

+ ... (Higher order terms)

We can then write:

$$f(x+u, y+v) \approx f(x, y) + uf_x(x, y) + vf_y(x, y)$$

Chapter 4. Edge Detection

4.4 Corner detection

Harris corner derivation

$$E(u, v) = \sum_{x,y} w(x, y) [f(x + u, y + v) - f(x, y)]^2$$

$$E(u, v) = \sum_{x,y} w(x, y) [f(x, y) + uf_x + vf_y - f(x, y)]^2$$

$$E(u, v) = \sum_{x,y} w(x, y) [uf_x + vf_y]^2 = \sum_{x,y} w(x, y) (u^2 f_x^2 + v^2 f_y^2 + uv + uv f_x f_y)$$

$$E(u, v) = [u \ v] \left(\sum_{x,y} w(x, y) \begin{bmatrix} f_x^2 & f_x f_y \\ f_x f_y & f_y^2 \end{bmatrix} \right) \begin{bmatrix} u \\ v \end{bmatrix} = [u \ v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

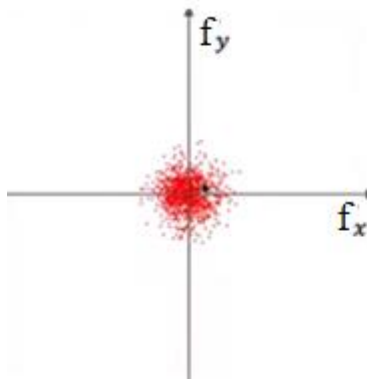
$E(u, v)$ is the equation of an ellipse, and M is the covariance matrix. We compute the eigen values of M : (λ_1, λ_2) .

The eigenvectors of a covariance matrix are the same as the directions of principle component (i.e. the axes of a ellipsoid enclosing the data points), and the eigenvalues measure the variances of date points along eigenvectors.

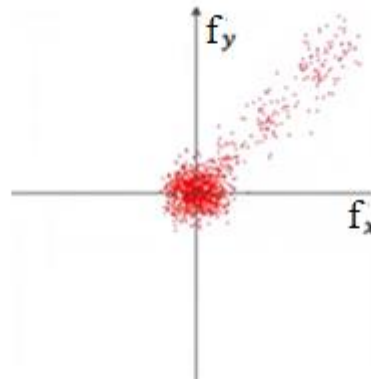
Chapter 4. Edge Detection

4.4 Corner detection

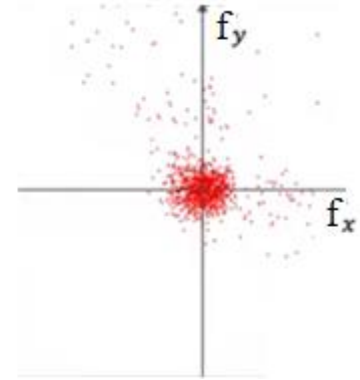
Distribution of image gradient



Flat region



Edge region

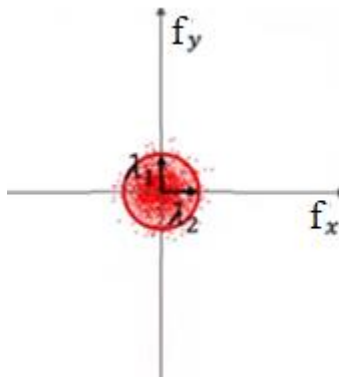


Corner region

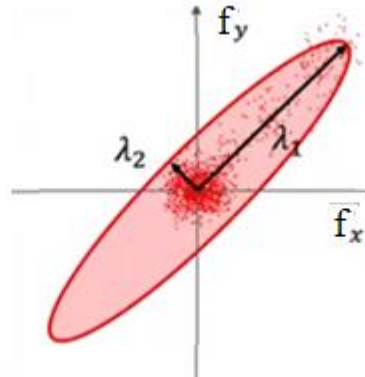
Chapter 4. Edge Detection

4.4 Corner detection

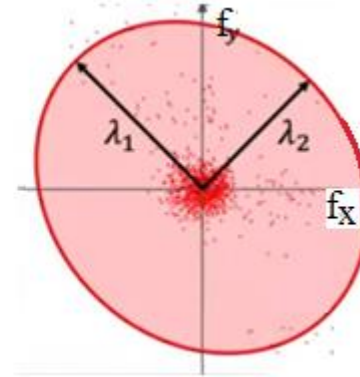
Distribution of image gradient
Finding elliptical disk of distribution



Flat region



Edge region

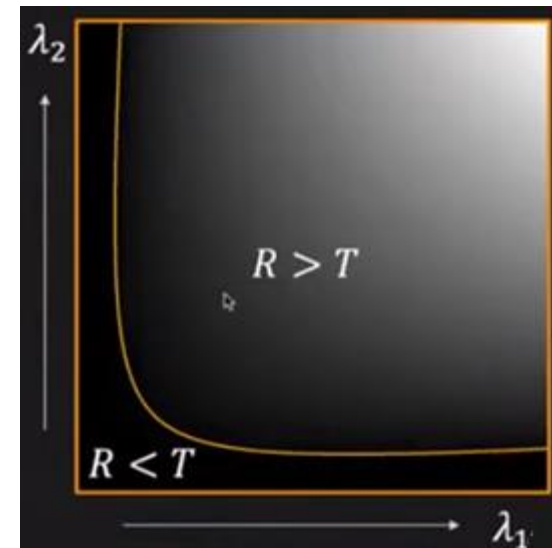
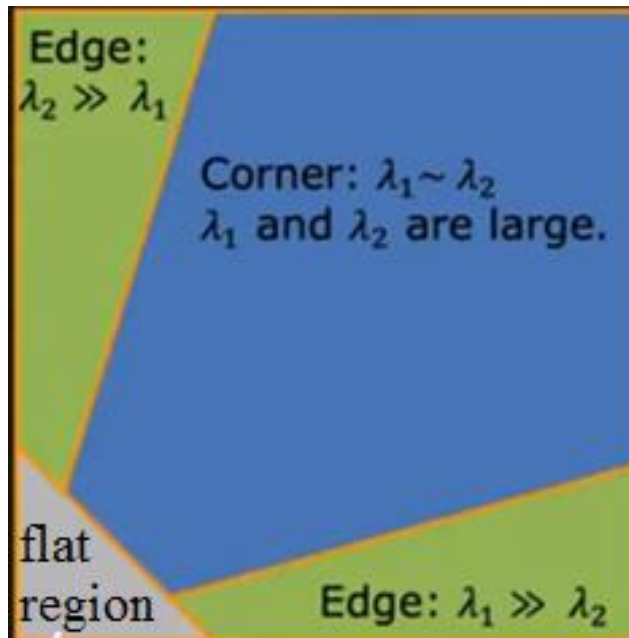


Corner region

Chapter 4. Edge Detection

4.4 Corner detection

Harris corner response function



$$R = \det M - k (\text{trace } M)^2$$

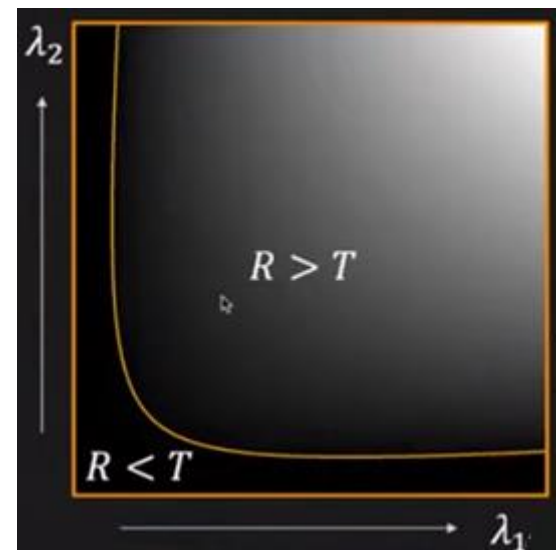
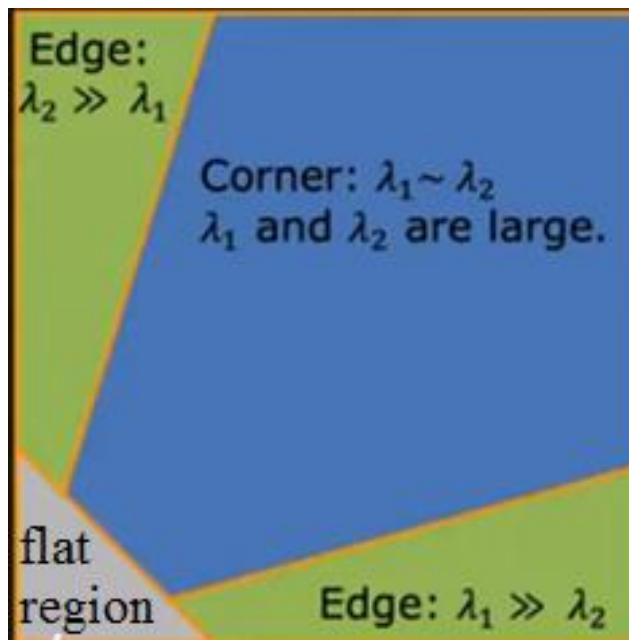
$K=0,04$ to $0,06$ (empirically determined)

$$\det M = \lambda_1 \lambda_2$$
$$\text{trace } M = \lambda_1 + \lambda_2$$

Chapter 4. Edge Detection

4.7 Corner detection

Harris corner response function



$$\det \mathbf{M} = m_{11}m_{22} - m_{21}m_{12}$$

$$\text{tr } \mathbf{M} = m_{11} + m_{22}$$

Chapter 4. Edge Detection

4.4 Corner detection

Harris corner response function

Harris & Stephens (1988)

$$R = \det(M) - \kappa \text{trace}^2(M)$$

Kanade & Tomasi (1994)

$$R = \min(\lambda_1, \lambda_2)$$

Nobel (1998)

$$R = \frac{\det(M)}{\text{trace}(M) + \epsilon}$$

Chapter 4. Edge Detection

4.4 Corner detection

Harris corner detection: Algorithm

1. Convert the original image into a grayscale image I . The pixel values of I are computed as a weighted sum of the corresponding R , G , B values:

$$I = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

2. Compute the derivatives I_x and I_y by **convolving** the image I with the **Sobel operator**:

$$I_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} * I \quad \text{and} \quad I_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * I$$

3. Compute the products of the derivatives $I_x I_x$, $I_x I_y$, $I_y I_y$.
4. Convolve the images $I_x I_x$, $I_x I_y$, $I_y I_y$ with a Gaussian filter or a mean filter.
5. Compute the response function for each pixel:

$$R = \det(M) - k \operatorname{tr}(M)^2.$$

6. Set a threshold T on the value of R and find pixels with responses above this threshold. Finally, compute the **non-max suppression** in order to pick up the optimal corners.